

UNIT 2

USER EXPERIENCE

User Interactions, User interface design principle.

User interaction and user interface (UI) design are crucial aspects of creating a successful Android application. The design principles focus on providing a positive and intuitive user experience. Here are some key principles for user interaction and UI design in Android:

1.Consistency:

Maintain consistency in terms of design elements, colors, fonts, and interactions throughout your application. Consistency helps users build a mental model of your app, making it more predictable and easier to navigate.

2.Clarity:

Ensure that your UI is clear and easy to understand. Use concise and familiar language, and avoid jargon or complex terminology. Clearly communicate the purpose of each screen and feature.

3.Feedback:

Provide visual and audio feedback to acknowledge user actions. This helps users understand that their input has been received and prevents confusion. For example, use animations, toast messages, or subtle transitions.

4.Efficiency:

Design with efficiency in mind. Minimize the number of steps users need to take to accomplish tasks. Use shortcuts, gestures, and smart defaults to streamline the user experience.

5.Learnability:

Make your app easy to learn for new users. Use standard Android patterns and conventions so that users can leverage their existing knowledge of the platform when using your app for the first time.

6.Hierarchy and Navigation:

Establish a clear hierarchy in your app's information architecture. Use consistent navigation patterns, such as the navigation drawer, tabs, or bottom navigation, to help users easily move between different sections of your app.

7.Accessibility:

Ensure that your app is accessible to users with different abilities. Follow Android's accessibility guidelines and provide options for users with visual, auditory, or motor impairments.

8.Aesthetics:

Pay attention to the visual design of your app. Use a cohesive color scheme, appropriate typography, and meaningful imagery to create a visually appealing and engaging interface.

9.Flexibility and Adaptability:

Design your app to work well on various screen sizes and resolutions. Consider both portrait and landscape orientations. Use responsive layouts and provide alternative resources for different screen densities.

10.User Control:

Allow users to have control over their experience. Provide options and settings that allow users to customize the app to their preferences, such as font size, theme, and notification preferences.

11.Error Handling:

Design clear error messages that guide users on how to correct issues. Prevent and handle errors gracefully, ensuring that users can recover from mistakes without frustration.

12.Performance:

Optimize your app for performance to provide a smooth and responsive user experience. Minimize loading times, use efficient animations, and prioritize content that users are likely to access frequently.

Testing of user Interface in Android:-

Testing the user interface (UI) of an Android application is a critical part of the development process to ensure a positive user experience. There are various approaches and tools available for testing Android UI, and here are some key aspects to consider:

1.Manual Testing:

Exploratory Testing: Manually explore the app's UI to identify usability issues, layout problems, and any inconsistencies.

Usability Testing: Have real users interact with the app to gather feedback on the overall usability and user satisfaction.

2.Automated Testing:

UI Automation Testing: Use tools like Espresso or UI Automator to automate UI interactions and validate the app's behavior. Espresso is particularly popular for testing the UI of Android applications.

UI Testing Frameworks: Leverage testing frameworks like Appium, Calabash, or Robot Framework for cross-platform testing, including Android UI testing.

3.Integration Testing:

Test UI Interactions with Backend: Ensure that UI interactions and data flow seamlessly between the app's UI and the backend services. Integration testing helps catch issues related to data synchronization.

4.Performance Testing:

UI Performance Testing: Evaluate the app's UI performance by measuring response times, screen transitions, and animations. Tools like Android Profiler can assist in identifying performance bottlenecks.

Memory Leak Detection: Use tools like LeakCanary to identify and fix memory leaks that could impact the app's performance and responsiveness.

5.Accessibility Testing:

Screen Reader Testing: Verify that the app is accessible to users with disabilities by testing with Android's built-in screen readers. Ensure that all UI components are labeled correctly and that navigation is possible using accessibility features.

Orientation and Resolution Testing:

Screen Size Testing: Test your app's UI on various screen sizes and resolutions, ensuring that it scales appropriately and maintains a consistent layout.

Orientation Testing: Verify that your app works correctly in both portrait and landscape orientations.

6.Localization Testing:

Language and Region Testing: Test the app with different languages and regions to ensure that UI elements, text, and date/time formats are displayed correctly.

7.Cross-Device Testing:

Device Fragmentation Testing: Test your app on a range of Android devices to account for hardware and software variations. Emulators and cloud-based testing services can assist in testing on multiple devices.

8.User Flow and Navigation Testing:

User Journey Testing: Test the app's user flow and navigation paths to ensure that users can easily accomplish their tasks. Identify any navigation or usability issues.

9.Security Testing:

UI Security Testing: Check for security vulnerabilities related to the UI, such as improper input validation, secure data transmission, and secure storage of sensitive information.

10.Continuous Integration (CI) and Continuous Deployment (CD):

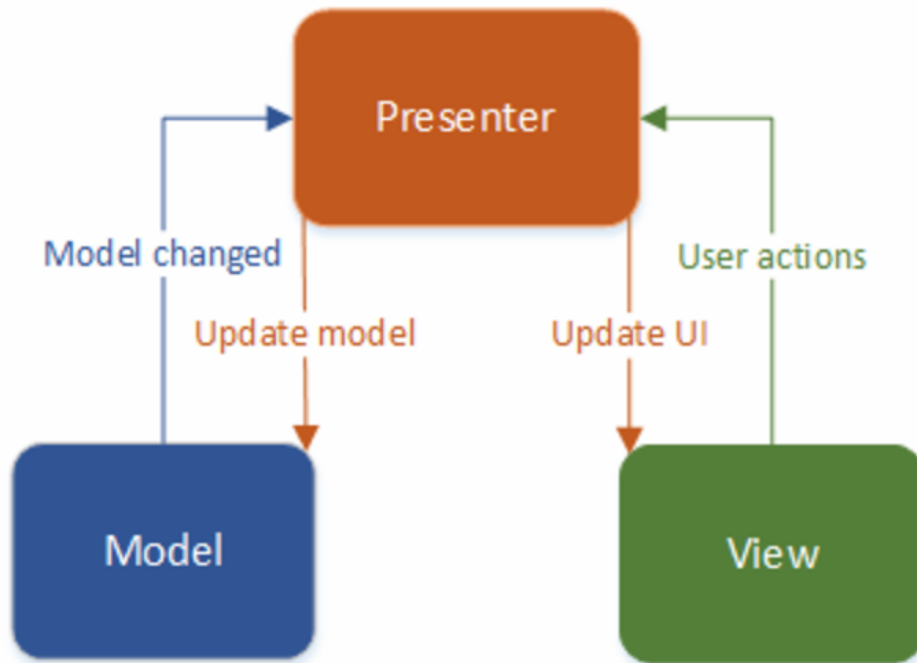
Automate UI Tests in CI/CD Pipelines: Integrate UI tests into your CI/CD pipelines to ensure that UI-related issues are identified early in the development process.

Model View Presenter in Android Studio.

Model-View-Presenter (MVP) is a software architectural pattern commonly used in Android development to separate concerns and improve the testability of the code. In the MVP pattern, the responsibilities are divided into three main components: Model, View, and Presenter.

Here's a brief overview of each component:

MVP



Model:

Represents the data and business logic of the application.

It should be independent of the UI and communicate with the Presenter to update or retrieve data.

View:

Represents the UI elements and is responsible for displaying data to the user.

Should be passive and not contain business logic.

Communicates with the Presenter to handle user input and update the UI based on the data received from the Presenter.

Presenter:

Acts as an intermediary between the Model and View.

Contains the application's business logic, handles user input from the View, and updates the Model and View accordingly.

Should not contain direct references to Android UI components to keep it independent and testable.

To implement the MVP pattern in Android Studio, you can follow these general steps:

Create Model:

Define your data classes and business logic. This could include network calls, database interactions, or any other data-related operations.

Create View:

Implement your Android UI components, layout files, and activities/fragments.

The View interfaces with the Presenter to update the UI and handle user input.

Create Presenter:

Implement the Presenter, which contains the application's business logic.

The Presenter communicates with the Model to retrieve or update data and with the View to update the UI.

Define Interfaces:

Create interfaces to define the contract between the View and Presenter. This helps in making the components loosely coupled and easily testable.

Example:-

```
// Model  
public class UserDataModel {  
    // Define your data and business logic here
```

```
}

// View
public interface UserView {
    void displayUserData(UserDataModel userData);
}

// Presenter
public class UserPresenter {
    private UserView view;
    private UserDataModel model;

    public UserPresenter(UserView view) {
        this.view = view;
        this.model = new UserDataModel();
    }

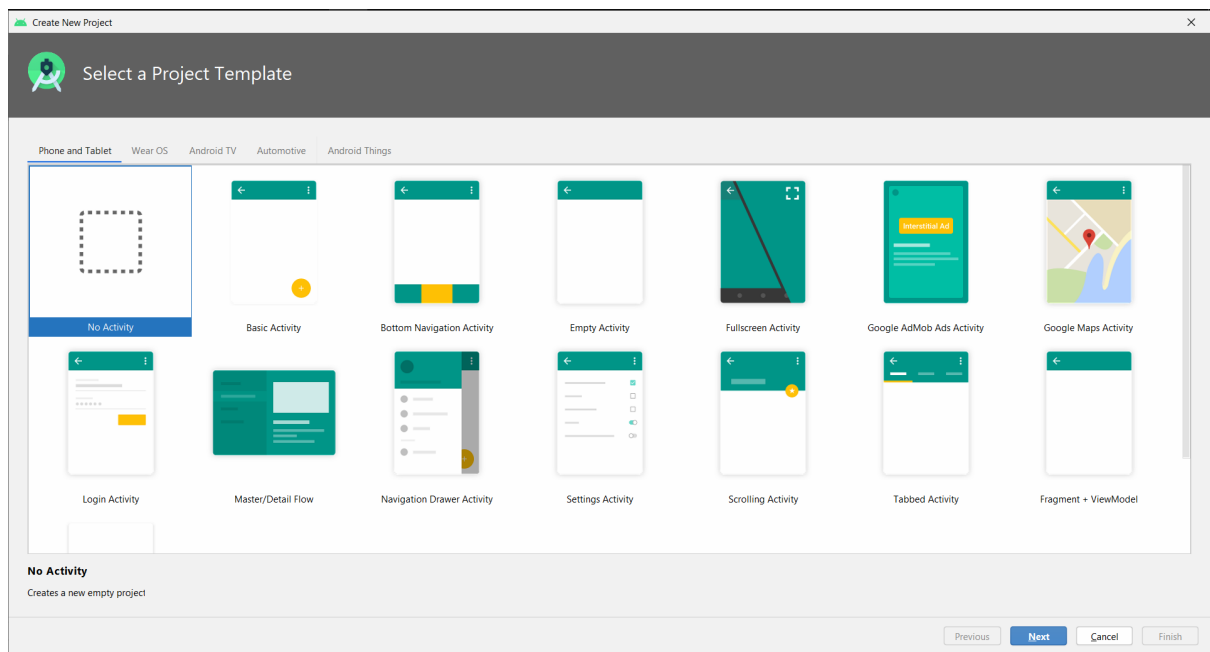
    public void fetchData() {
        // Business logic to fetch data from the model
        UserDataModel userData = model.getData();

        // Update the view with the retrieved data
        view.displayUserData(userData);
    }
}
```

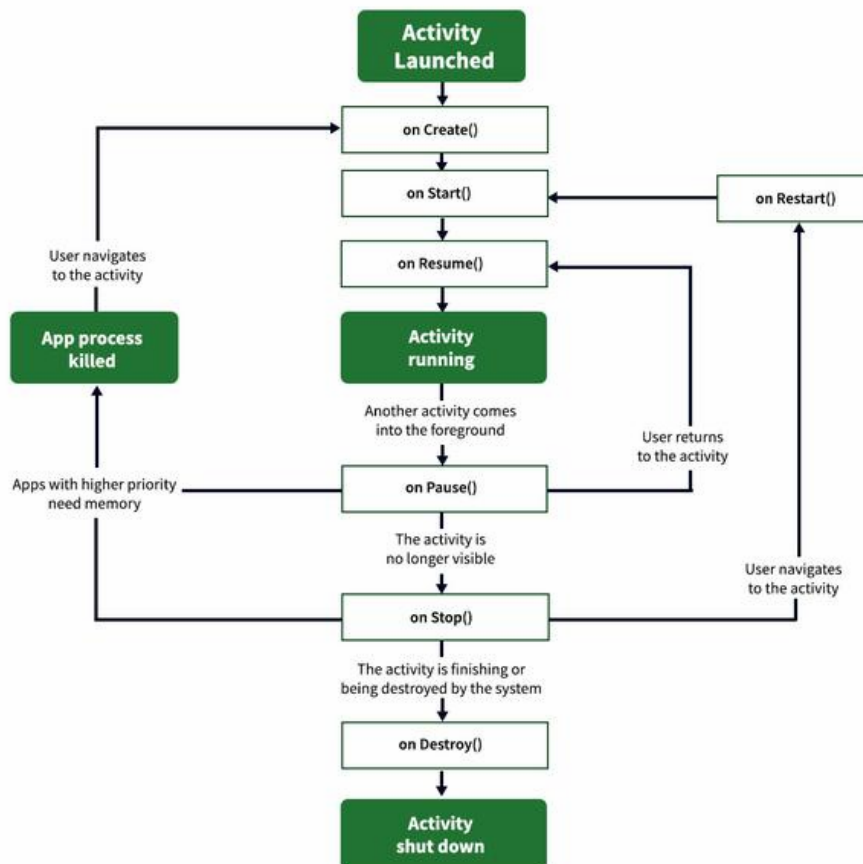
In your Android activity or fragment, you would instantiate the Presenter and pass the View to it. The View would then call methods on the Presenter to interact with the data.

Activities in Android:-

- An **Activity** is a component that provides a screen where users can interact with the app.
- Each Activity is a subclass of the `android.app.Activity` class.
- Example: A login screen, a contacts list, or a photo viewer can all be different Activities.



Activity LifeCycle



Activity Lifecycle in Android

1. onCreate()

It is called when the activity is first created. This is where all the static work is done like creating views, binding data to lists, etc. This method also provides a Bundle containing its previous frozen state, if there was one.

2. onStart()

It is invoked when the activity is visible to the user. It is followed by onResume() if the activity is invoked from the background. It is also invoked after onCreate() when the activity is first started.

3. onRestart()

It is invoked after the activity has been stopped and prior to its starting stage and thus is always followed by onStart() when any activity is revived from background to on-screen.

4. onResume()

It is invoked when the activity starts interacting with the user. At this point, the activity is at the top of the activity stack, with a user interacting with it. Always followed by onPause() when the activity goes into the background or is closed by the user

5. onPause()

It is invoked when an activity is going into the background but has not yet been killed. It is a counterpart to onResume(). When an activity is launched in front of another activity, this callback will be invoked on the top activity (currently on screen). The activity, under the active activity, will not be created until the active activity's onPause() returns, so it is recommended that heavy processing should not be done in this part.

6. onStop()

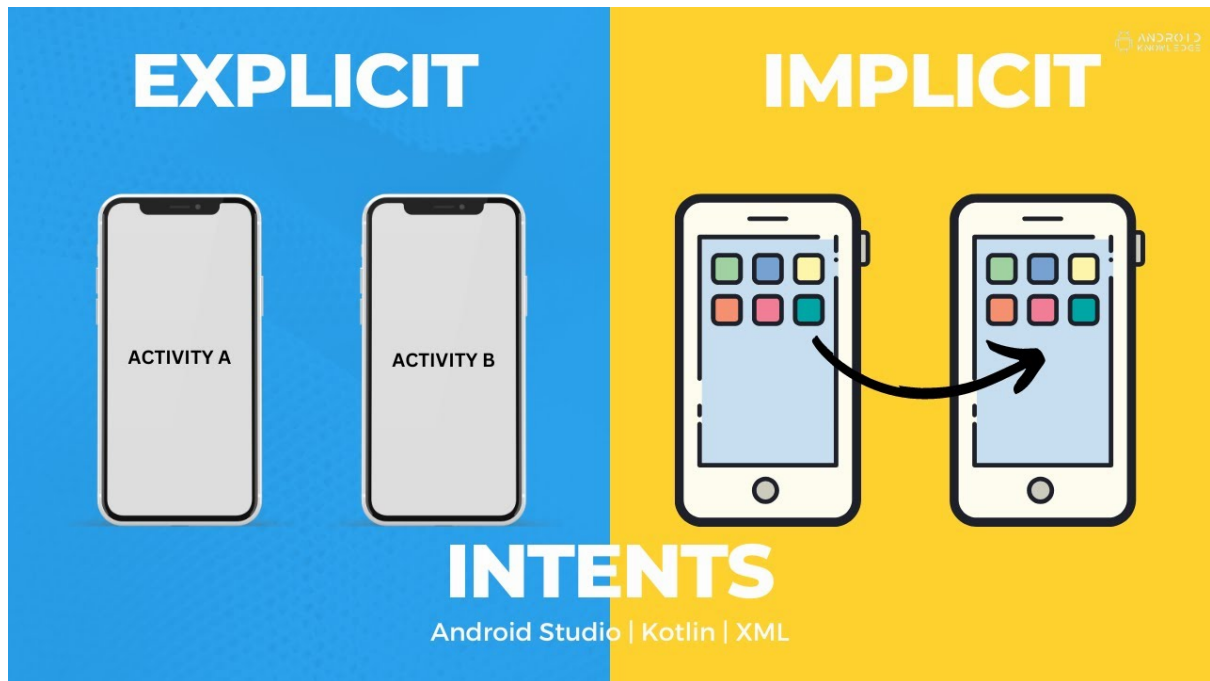
It is invoked when the activity is not visible to the user. It is followed by **onRestart()** when the activity is revoked from the background, followed by onDestroy() when the activity is closed or finished, and nothing when the activity remains on the background only. Note that this method may never be called, in low memory situations where the system does not have enough memory to keep the activity's process running after its onPause() method is called.

7. onDestroy()

The final call received before the activity is destroyed. This can happen either because the activity is finishing (when finish() is invoked) or because the system is temporarily destroying this instance of the activity to save space. To distinguish between these scenarios, check it with **isFinishing()** method.

Intents in Android

This process of taking users from one application to another is achieved by passing the **Intent** to the system. **Intents**, in general, are used for navigating among various activities within the same application, but note, is not limited to one single application, i.e., they can be utilized from moving from one application to another as well.



Types of Android Intents

There are two types of intents in android

1. Implicit
1. Explicit

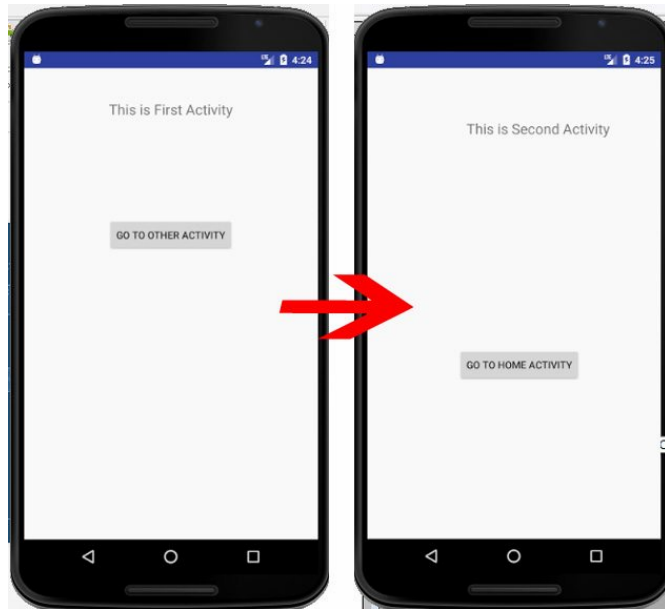
Implicit Intent

Implicit Intent doesn't specify the component. In such a case, intent provides information on available components provided by the system that is to be invoked. For example, you may write the following code to view the webpage.

```
Intent intent=new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("URL"));  
startActivity(intent);
```

Explicit Intent

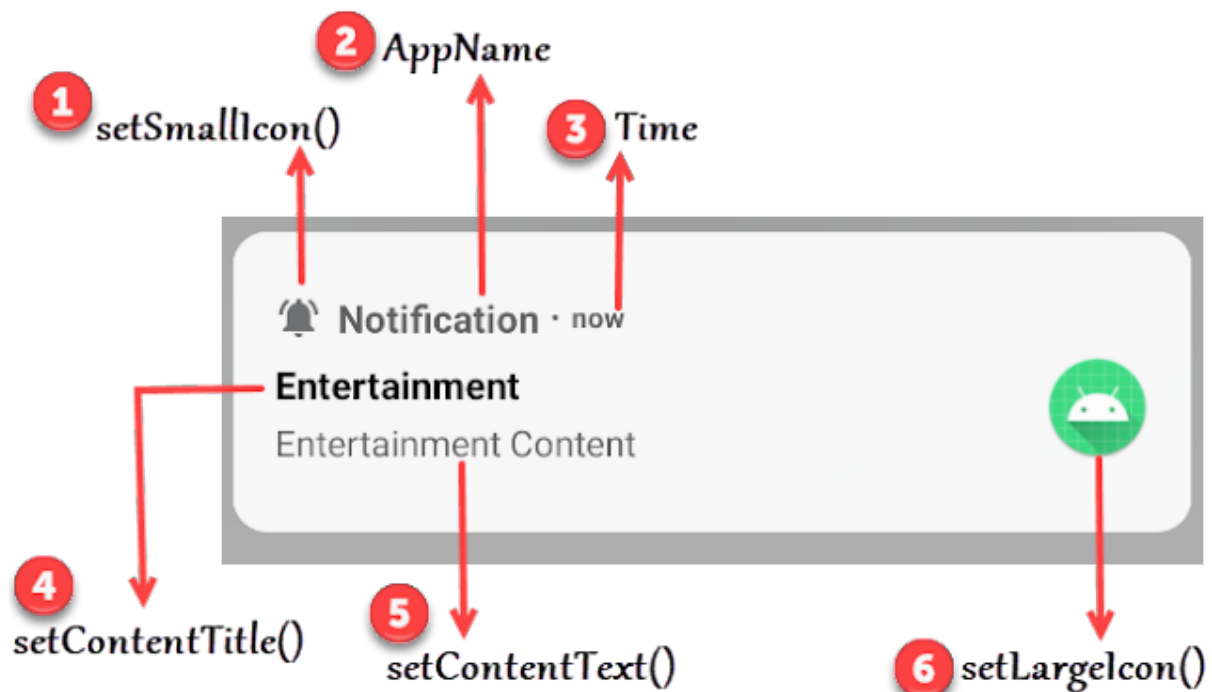
Explicit Intent specifies the component. In such a case, intent provides the external class to be invoked.



Notification

Notifications provide short, timely information about events in your app while it's not in use

A notification is a message that Android displays outside your app's UI to provide the user with reminders, communication from other people, or other timely information from your app. Users can tap the notification to open your app or take an action directly from the notification.



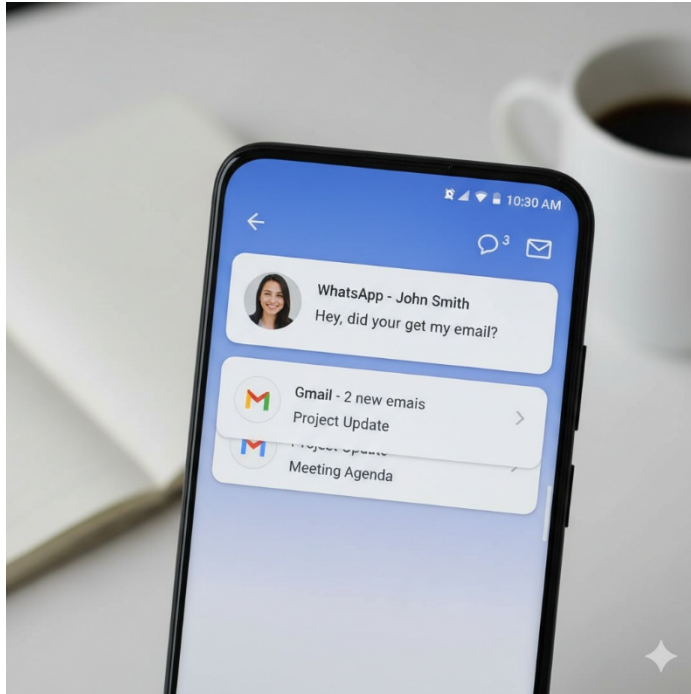
The most common parts of a notification are indicated above diagram as follows:

1. **SmallIcon** : This is set by calling `setSmallIcon()` .
2. **App Name** : This is by default provided by system .
3. **Time** : This is by default provided by system , It can be overridden by calline `setWhen()` or hide by calling `setShowWhen(false)` .
4. **Title** : This is set by calling `setContentTitle()` .
5. **Content** : This is set by calling `setContentText()` .
6. **LargeIcon** : This is set by calling `setLargeIcon()` .

Types of Notifications in Android (MAD context):

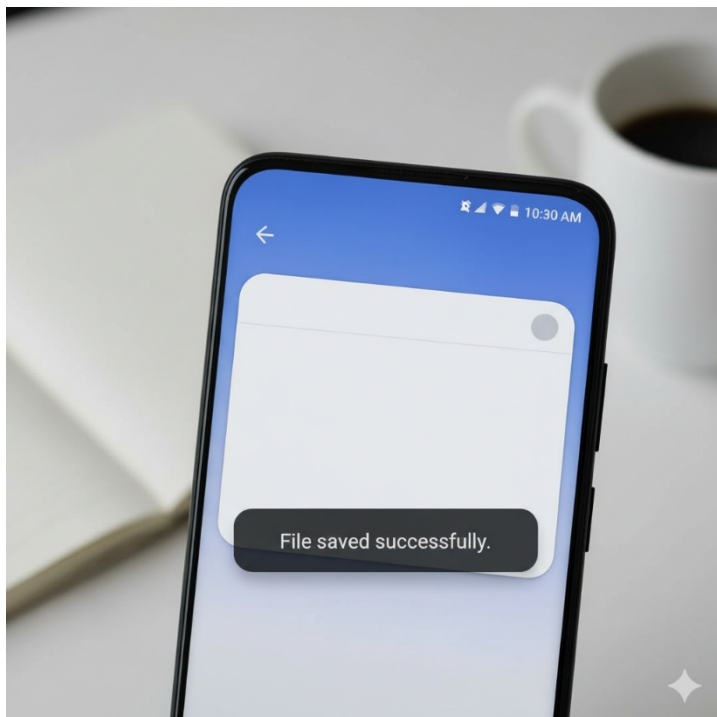
1. Status Bar Notifications

- Appear in the notification drawer and status bar.
- Example: New message received, email updates.



2. Toast Notifications

- Small, transient popup messages that disappear automatically after a short duration.
- Example: “Message sent” or “File saved successfully.”



3. Dialog Notifications

- Displayed as a dialog box over the current activity.
- Example: Low battery warning, permission request.



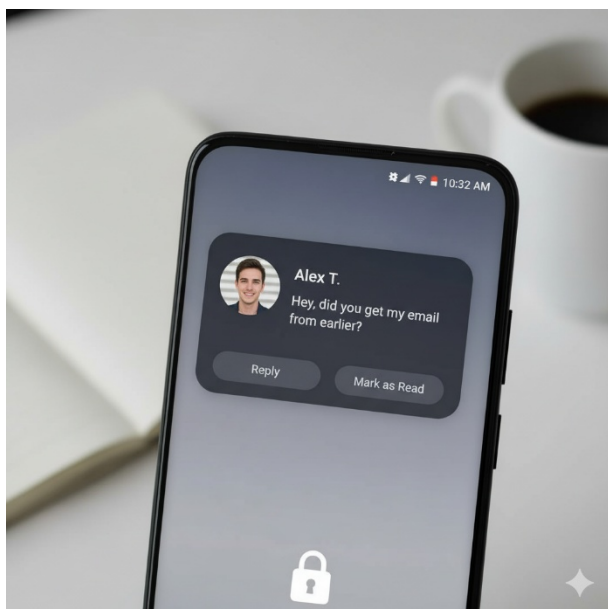
4. Heads-Up Notifications (High Priority)

- Appear as a floating popup over the current screen for immediate attention.
- Example: Incoming call, alarm alert.



5. Push Notifications (Remote Notifications)

- Sent from a server to the device using services like Firebase Cloud Messaging (FCM).
- Example: WhatsApp message alert, social media updates.



Notification Channel

- Starting in Android 8.0 (API level 26), all notifications must be assigned to a channel or it will not appear.
- Categorizing notifications into channels User can disable notification for particular category instead of all

Android Manifest File (AndroidManifest.xml)

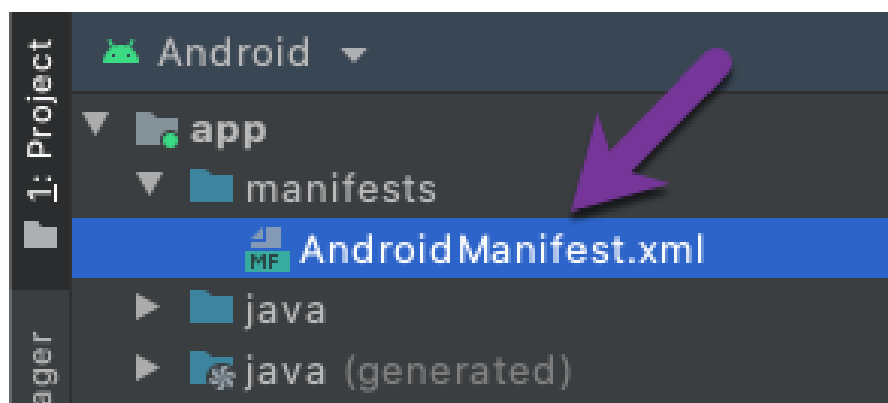
The AndroidManifest.xml file is the heart of an Android app.

It provides essential information about your app to the Android system, such as:

- App identity (name, icon, version).
- Components (activities, services, receivers, providers).
- Permissions needed.
- Hardware/software features used.

It's located in every Android project at:

app/src/main/AndroidManifest.xml



```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp"> [ROOT tag]
```

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.MyApp"> [Application tag]

    <activity android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity> [Activity on screen ]

</application>

</manifest>
```

LAYOUTS:- LINEAR and RELATIVE

LinearLayout VS. RelativeLayout

LinearLayout



RelativeLayout



Layouts in Android define how UI elements (Views/Widgets) are arranged on the screen.

Two commonly used layouts are:

1. LinearLayout
2. RelativeLayout

LinearLayout

- Arranges child views in a **single direction** → **vertical** or **horizontal**.
- Children are placed one after another in a **row** or **column**.
- You can use **android:orientation="vertical"** or **horizontal**.

<LinearLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:orientation="vertical"
```

```
android:padding="16dp">
```

```
<TextView
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Username" />
```

```
<EditText
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:hint="Enter Username" />
```

```
<Button
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Login" />
```

```
</LinearLayout>
```

Horizontal LinearLayout

```
<LinearLayout
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="horizontal"
```

```
    android:gravity="center">
```

```
<Button
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Yes" />
```

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="No" />
```

```
</LinearLayout>
```

RelativeLayout

- Positions child views **relative to each other** or to the **parent container**.
- More flexible than LinearLayout.
- Uses attributes like:
 - android:layout_alignParentTop="true"
 - android:layout_below="@id/someView"
 - android:layout_toRightOf="@id/someView"

Example: RelativeLayout

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">
```

```
<TextView
    android:id="@+id/label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Username" />
```

```
<EditText
```

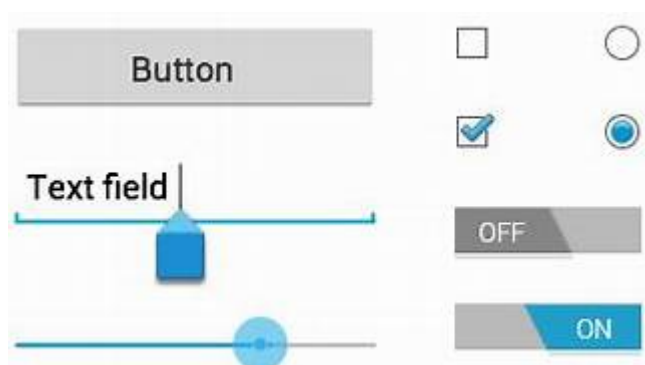
```
android:id="@+id/username"  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:layout_toEndOf="@id/label"  
android:layout_marginStart="16dp"  
android:hint="Enter Username" />
```

<Button

```
android:id="@+id/loginButton"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:layout_below="@id/username"  
android:layout_centerHorizontal="true"  
android:layout_marginTop="20dp"  
android:text="Login" />
```

</RelativeLayout>

Android User Input Controls : Button, Text Field, Seek bar, Checkbox, Radio Button, Toggle Button



1. Button

Definition

- A Button is a clickable UI element used to perform an action when tapped (e.g., submit form, open a new screen).

Common Attributes

- android:text → Sets button label.
- android:onClick → Directly link to a function in Activity/Fragment.
- android:enabled → Enable/disable interaction.
- android:backgroundTint → Change background color.

Example

<Button

```
    android:id="@+id/btnLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login" />
```

2. Text Field (EditText)

Definition

- An EditText allows users to input text (name, email, password, etc.).

Common Attributes

- android:hint → Placeholder text.
- android:inputType → Defines type of input (e.g., text, number, textPassword, textEmailAddress).
- android:maxLines → Limit number of lines.
- android:ems → Width in terms of character units.

Example

<EditText

```
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Email"
    android:inputType="textEmailAddress" />
```


3. SeekBar

Definition

- A SeekBar is a slider that allows users to pick a value from a continuous range (e.g., volume control, brightness).

Common Attributes

- android:max → Maximum value (default: 100).
- android:progress → Current value.
- android:secondaryProgress → For buffer indication.

Example

<SeekBar

```
    android:id="@+id/seekVolume"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100"
    android:progress="50" />
```

4. CheckBox

Definition

- A CheckBox allows multiple independent selections (e.g., “Accept terms”, “Receive notifications”).

Common Attributes

- android:text → Label text.
- android:checked → Default checked/unchecked.

Example

<CheckBox

```
    android:id="@+id/cbSubscribe"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Subscribe to newsletter"
```

```
android:checked="false" />
```

5. RadioButton (with RadioGroup)

Definition

- RadioButton lets users select one option from a group.
- Must be placed inside a RadioGroup to enforce mutual exclusivity.

Common Attributes

- android:text → Option label.
- android:checked → Default checked option.

Example

```
<RadioGroup
```

```
    android:id="@+id/rgGender"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```

```
<RadioButton
```

```
    android:id="@+id/rbMale"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Male" />
```

```
<RadioButton
```

```
    android:id="@+id/rbFemale"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Female" />
```

```
</RadioGroup>
```

6. ToggleButton / Switch

Definition

- A ToggleButton switches between two states (ON/OFF).
- In Material Design, prefer **Switch** (SwitchMaterial).

Common Attributes

- android:textOn → Text when ON.
- android:textOff → Text when OFF.
- android:checked → Initial state.

Example

<ToggleButton

android:id="@+id/toggleWifi"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:textOn="Wi-Fi ON"

android:textOff="Wi-Fi OFF" />